

Declaring non-type template parameters with **auto**

Document number: P0127R2
Date: 2016-06-23
Project: Programming Language C++, Evolution Working Group
Revises: [P0127R1](#)
Reply-to: James Touton <bekenn@gmail.com>
Mike Spertus, Symantec <mike_spertus@symantec.com>

Table of Contents

1. [Table of Contents](#)
2. [What's New](#)
3. [Introduction](#)
4. [Feature Test Macro](#)
5. [Wording](#)
6. [Acknowledgments](#)

What's New

Changes since P0127R1

- Updated standardese and removed detailed design overview.

Introduction

This paper proposes allowing non-type template parameters to be declared with the **auto** placeholder type specifier. For a detailed discussion of the feature and design decisions, see [P0127R1](#).

Example:

```
template <auto v> struct S; // type of v is deduced
```

Feature Test Macro

The recommended feature test macro is `__cpp_template_auto`.

Wording

All modifications are presented relative to N4594.

Modify §7.1.6.4 [dcl.spec.auto] paragraph 5:

A placeholder type can also be used in declaring a variable in the *condition* of a selection statement (6.4) or an iteration statement (6.5), in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (5.3.4), in a *for-range-declaration*, and in declaring a static data member with a *brace-or-equal-initializer* that appears within the *member-specification* of a class definition (9.4.2), and as a *decl-specifier of the parameter-declaration's decl-specifier-seq in a template-parameter* (14.1).

Delete §7.1.6.4 [dcl.spec.auto] paragraph 7. (The rules specified in this paragraph are reformulated below as §7.1.6.4.1 [dcl.auto.deduct])

When a variable declared using a placeholder type is initialized, or a return statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type or variable type is determined from the type of its initializer. [...]

Modify §7.1.6.4 [dcl.spec.auto] paragraph 8 (now paragraph 7):

If the *init-declarator-list* contains more than one *init-declarator*, they shall all form declarations of variables. The type of each declared variable is determined as described above by placeholder type deduction (7.1.6.4.1), and if the type that replaces the placeholder type is not the same in each deduction, the program is ill-formed.

[Example:

```
auto x = 5, *y = &x;           // OK: auto is int
auto a = 5, b = { 1, 2 };      // error: different types for auto
```

—end example]

Add a new section §7.1.6.4.1 [dcl.auto.deduct] with paragraphs as follows (highlighted text denotes meaningful changes from the deleted §7.1.6.4 paragraph 7):

Placeholder type deduction is the process by which a type containing a placeholder type is replaced by a deduced type.

A type τ containing a placeholder type, and a corresponding initializer e , are determined as follows:

- for a `return` statement that occurs in a function declared with a return type that contains a placeholder type, τ is the declared return type and e is the operand of the return statement. If the `return` statement has no operand, then e is `void{}`;
- for a variable declared with a type that contains a placeholder type, τ is the declared type of the variable and e is the initializer. If the initialization is direct-list-initialization, the initializer shall be a *braced-init-list* containing only a single *assignment-expression* and e is the *assignment-expression*;
- for a non-type template parameter declared with a type that contains a placeholder type, τ is the declared type of a non-type template parameter and e is a corresponding template argument.

In the case of a `return` statement with no operand or with an operand of type `void`:

- if the placeholder is `decltype(auto)`, then τ shall be `decltype(auto)`;
- otherwise, τ shall be `cv auto`.

If the deduction is for a `return` statement and e is a *braced-init-list* (8.5.4), the program is ill-formed.

If the placeholder is the `auto` *type-specifier*, the deduced type τ' replacing τ is determined using the rules for template argument deduction. Obtain P from τ by replacing the occurrences of `auto` with either a new invented type template parameter U or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`. Deduce a value for U using the rules of template argument deduction from a function call (14.8.2.1), where P is a function template parameter type and the corresponding argument is e . If the deduction fails, the declaration is ill-formed. Otherwise, τ' is obtained by substituting the deduced U into P . [Example:

```
auto x1 = { 1, 2 };           // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 };         // error: cannot deduce element type
auto x3{ 1, 2 };              // error: not a single element
auto x4 = { 3 };              // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };                 // decltype(x5) is int
```

—end example]

[Example:

```
const auto& i = expr;
```

The type of `i` is the deduced type of the parameter u in the call `f(expr)` of the following invented function template:

```
template <class U> void f(const U& u);
```

—end example]

If the placeholder is the `decltype(auto)` *type-specifier*, τ shall be the placeholder alone. The type deduced for τ is

determined as described in 7.1.6.2, as though `e` had been the operand of the `decltype`. [*Example:*

```
int i;
int&& f();
auto          x2a(i);          // decltype(x2a) is int
decltype(auto) x2d(i);          // decltype(x2d) is int
auto          x3a = i;          // decltype(x3a) is int
decltype(auto) x3d = i;          // decltype(x3d) is int
auto          x4a = (i);         // decltype(x4a) is int
decltype(auto) x4d = (i);         // decltype(x4d) is int&
auto          x5a = f();         // decltype(x5a) is int
decltype(auto) x5d = f();         // decltype(x5d) is int&&
auto          x6a = { 1, 2 };    // decltype(x6a) is std::initializer_list<int>
decltype(auto) x6d = { 1, 2 };    // error, { 1, 2 } is not an expression
auto*          x7a = &i;         // decltype(x7a) is int*
decltype(auto)* x7d = &i;         // error, declared type is not plain decltype(auto)
```

—end example]

Modify §14.1 [temp.param] paragraph 4:

A non-type *template-parameter* shall have one of the following (optionally *cv-qualified*) types:

- integral or enumeration type,
- pointer to object or pointer to function,
- lvalue reference to object or lvalue reference to function,
- pointer to member,
- `std::nullptr_t`,
- a type that contains a placeholder type (7.1.6.4).

Insert a new paragraph before §14.3.2 [temp.arg.nontype] paragraph 1:

If the type of a *template-parameter* contains a placeholder type (7.1.6.4, 14.1), the deduced parameter type is determined from the type of the *template-argument* by placeholder type deduction (7.1.6.4.1). If a deduced parameter type is not permitted for a *template-parameter* declaration (14.1), the program is ill-formed.

Modify §14.3.2 [temp.arg.nontype] paragraph 2:

[*Example:*

```
template<const int* pci> struct X { /* ... */ };
int ai[10];
X<ai> xi; // array to pointer and qualification conversions

struct Y { /* ... */ };
template<const Y& b> struct Z { /* ... */ };
Y y;
Z<y> z; // no conversion, but note extra cv-qualification

template<int (&pa)[5]> struct W { /* ... */ };
int b[5];
W<b> w; // no conversion

void f(char);
void f(int);

template<void (*pf)(int)> struct A { /* ... */ };
A<&f> a; // selects f(int)

template<auto n> struct B { /* ... */ };
B<5> b1; // OK: template parameter type is int
B<'a'> b2; // OK: template parameter type is char
B<2.5> b3; // error: template parameter type cannot be double
```

—end example]

Remove the original §14.5.5.1 [temp.class.spec.match] paragraph 3 (this paragraph is superfluous):

A non-type template argument can also be deduced from the value of an actual template argument of a non-type parameter of the primary template. [*Example: the declaration of a2 above. —end example*]

Remove §14.5.5 [temp.class.spec] paragraph 8 list item 8.1 from the bulleted list and insert it as a new paragraph after 14.5.5.1 [temp.class.spec.match] paragraph 3 with the modifications shown:

~~Each *template-parameter* shall appear at least once in the *template-id* outside a non-deduced context.~~ If the template arguments of a partial specialization cannot be deduced because of the structure of its *template-parameter-list* and the *template-id*, the program is ill-formed. [*Example:*

```
template <int I, int J> struct A {};
template <int I> struct A<I+5, I*2> {};    // error

template <int I> struct A<I, I> {};        // OK

template <int I, int J, int K> struct B {};
template <int I> struct B<I, I*2, 2> {};    // OK
```

—end example]

Modify §14.6.2.2 [temp.dep.expr] paragraph 3:

An *id-expression* is type-dependent if it contains

- an *identifier* associated by name lookup with one or more declarations declared with a dependent type,
- an *identifier* associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type (7.1.6.4).
- an *identifier* associated by name lookup with one or more declarations of member functions of the current instantiation declared with a return type that contains a placeholder type ~~(7.1.6.4),~~
- the *identifier* `__func__` (8.4.1), where any enclosing function is a template, a member of a class template, or a generic lambda,
- a *template-id* that is dependent,
- a *conversion-function-id* that specifies a dependent type, or
- a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization;

or if it names a dependent member of the current instantiation that is a static data member of type “array of unknown bound of T” for some T (14.5.1.3). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

```
simple-type-specifier ( expression-listopt )
::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt ( type-id ) new-initializeropt
dynamic_cast < type-id > ( expression )
static_cast < type-id > ( expression )
const_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
( type-id ) cast-expression
```

Modify §14.5.5.2 [temp.class.order] paragraph 1:

For two class template partial specializations, the first is more specialized than the second if, given the following rewrite to two function templates, the first function template is more specialized than the second according to the ordering rules for function templates (14.5.6.2):

- ~~the first~~ each of the two function templates ~~has the same template parameters as the first~~ its corresponding partial specialization ~~and. Each function template also has a single function parameter whose type is a class template specialization with where the template arguments of the first partial specialization are the corresponding template parameters from the function template for each template argument in the template-argument-list of the simple-template-id of the partial specialization, and.~~
- the second function template has the same template parameters as the second partial specialization and has a

single function parameter whose type is a class template specialization with the template arguments of the second partial specialization.

Modify §14.5.5.2 [temp.class.order] paragraph 2:

[Example:

```
template <int I, int J, class T> class X { };
template <int I, int J>          class X<I, J, int> { }; // #1
template <int I>                class X<I, I, int> { }; // #2

template <int I0, int J0> void f(X<I0, J0, int>); // A
template <int I0>        void f(X<I0, I0, int>); // B

template <auto v>         class Y { };
template <auto* p>        class Y<p> { }; // #3
template <auto** pp>      class Y<pp> { }; // #4

template <auto* p0>       void g(Y<p0>); // C
template <auto** pp0>    void g(Y<pp0>); // D
```

The partial specialization #2 is more specialized than the partial specialization #1 because According to the ordering rules for function templates, the function template B is more specialized than the function template A according to the ordering rules for function templates and the function template D is more specialized than the function template C. Therefore, the partial specialization #2 is more specialized than the partial specialization #1 and the partial specialization #4 is more specialized than the partial specialization #3. —end example]

Modify §14.5.6.2 [temp.func.order] paragraph 3:

To produce the transformed template, for each type, non-type, or template template parameter (including template parameter packs (14.5.3) thereof) synthesize a unique type, value, or class template respectively and substitute it for each occurrence of that parameter in the function type of the template. [Note: the type replacing the placeholder in the type of the value synthesized for a non-type template parameter is also a unique synthesized type. —end note]

Modify §14.8.2.5 [temp.deduct.type] paragraph 2:

In some cases, the deduction is done using a single set of types P and A, in other cases, there will be a set of corresponding types P and A. Type deduction is done independently for each P/A pair, and the deduced template argument values are then combined. If type deduction cannot be done for any P/A pair, or if for any pair the deduction leads to more than one possible set of deduced values, or if different pairs yield different deduced values, or if any template argument remains neither deduced nor explicitly specified, template argument deduction fails. The type of a type parameter is only deduced from an array bound if it is not otherwise deduced.

Replace §14.8.2.5 [temp.deduct.type] paragraph 13 with the following paragraph:

A template type argument cannot be deduced from the type of a non-type *template argument*.

When the value of the argument corresponding to a non-type template parameter P that is declared with a dependent type is deduced from an expression, the template parameters in the type of P are deduced from the type of the value. [Example:

```
template <long n> struct A { };

template <class T> struct C;
template <class T, T n> struct C<A<n>>
{
    using Q = T;
};

typedef long R;
typedef C<A<2>>::Q R; // OK; T was deduced to long from the template argument value in the type A<2>
```

—end example]

The type of N in the type $T[N]$ is `std::size_t`. [Example:

```
template <typename T> struct S;
template <typename T, T n> struct S<int[n]> {
    using Q = T;
};

typedef S<int[42]>::Q V;
typedef decltype(sizeof 0) V; // OK; T was deduced to std::size_t from the type int[42]
```

—end example]

Modify §14.8.2.5 [temp.deduct.type] paragraph 14:

[Example:

```
template<class T, T i> void f(int a[10](&a)[i]);
int v[10]{20};
f(v); // error: argument for template-parameter T cannot be deduced OK: T is std::size_t
```

—end example]

Insert a new section "Clause 14: templates" [diff.cpp14.temp] before §C.4.5 [diff.cpp14.string]:

14.8.2.5 [temp.deduct.type]

Change: Allowance to deduce from the type of a non-type template argument.

Rationale: In combination with the ability to declare non-type template arguments with placeholder types, allows partial specializations to decompose from the type deduced for the non-type template argument.

Effect on original feature: Valid C++ 2014 code may fail to compile or produce different results in this International Standard:

```
template <int N> struct A;
template <typename T, T N> int foo(A<N> *) = delete;
void foo(void *);
void bar(A<0> *p) {
    foo(p); // ill-formed; previously well-formed
}
```

Acknowledgments

Numerous people gave constructive feedback regarding the use of **auto** in template parameter lists in an [isocpp.org](https://ericniebler.com/2016/05/20/auto-in-template-parameter-lists/) discussion [thread](#).

Special thanks to Mike Spertus and Gabriel dos Reis for their invaluable analysis and assistance.

Thanks also to Andrew Sutton for assistance and instruction in identifying potential areas of conflict with the Concepts TS, and to Hubert Tong for essential contributions to the wording.